# Experiment 9
# Form Validation using HTML, CSS and JavaScript

**Name : Rayvan Sanjai KS**

**Roll no : 240701425**

## Aim

Todesign input formsthatvalidatedatasuchasemailandphonenumber,anddisplayerror messages using HTML, CSS and JavaScript with Validator.js.

## Step 1: Setting Up the HTML Form

CreateanHTMLform withinput fields forFull Name, Email, Phone Number, and Password. The novalidate attribute disables the browser's built-in validation so that JavaScript handles it instead.

## HTML Code

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <title>Form Validation</title>
</head>
<body>
  <form id="validationForm" novalidate>

    <div class="form-group">
      <label for="fullName">Full Name</label>
      <input type="text" id="fullName" placeholder="Enter your name" />
      <span class="error-msg" id="nameError"></span>
    </div>

    <div class="form-group">
      <label for="email">Email Address</label>
      <input type="email" id="email" placeholder="you@example.com" />
      <span class="error-msg" id="emailError"></span>
    </div>

    <div class="form-group">
      <label for="phone">Phone Number</label>
      <input type="tel" id="phone" placeholder="+91 98765 43210" />
      <span class="error-msg" id="phoneError"></span>
    </div>

    <div class="form-group">
      <label for="password">Password</label>
      <input type="password" id="password" placeholder="Min 8 characters" />
```

```
    <span class="error-msg" id="passwordError"></span>
  </div>

  <button type="submit">Submit</button>
 </form>
</body>
</html>
```

### Output Screen – Step 1:

A plain, unstyled form is rendered in the browser with four input fields and a submit button using default browser styling. No visual feedback or error messages appear yet.

## Step 2: Styling the Form with CSS

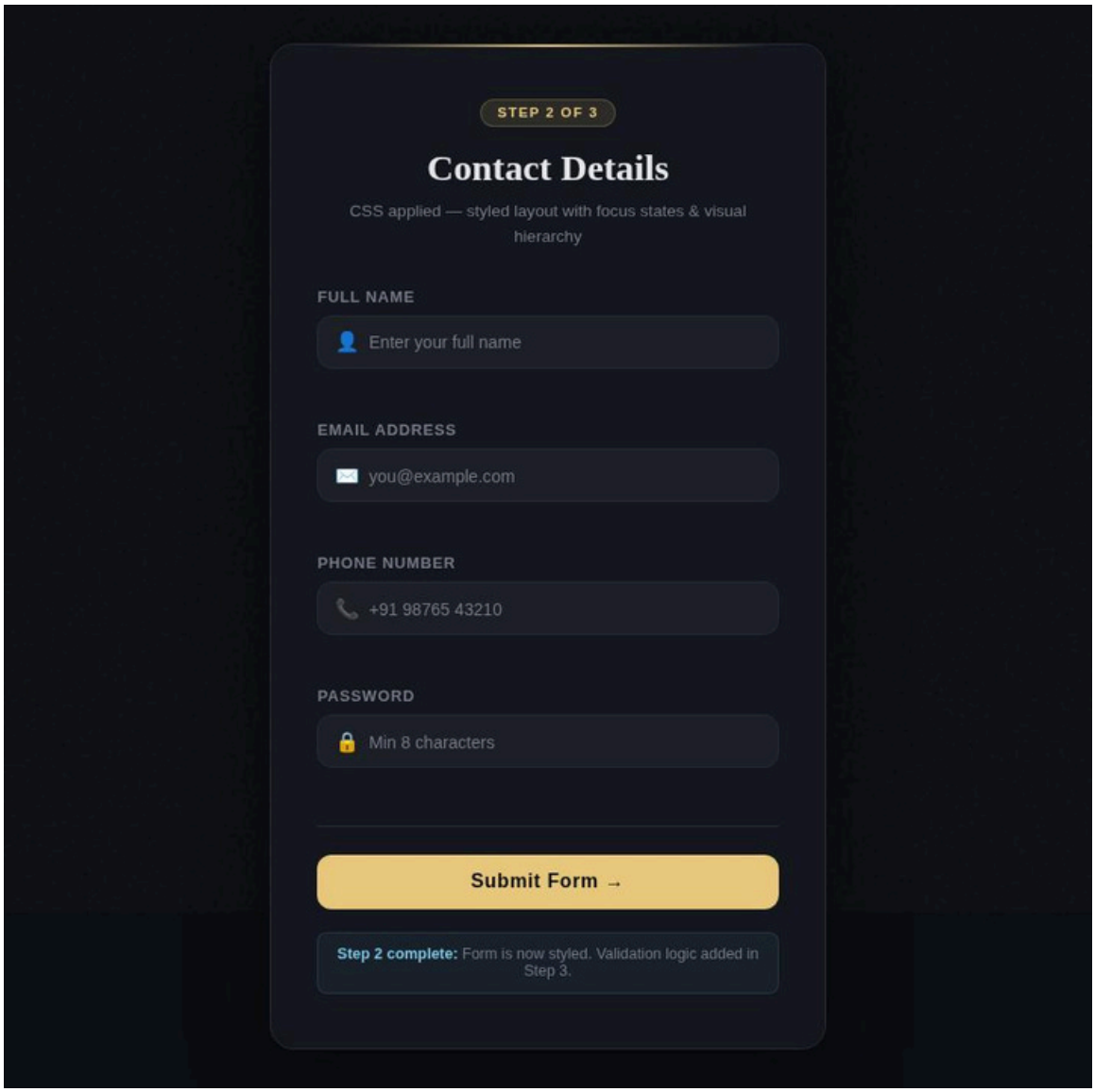
Add CSS to style the form with a clean card layout, styled input fields, focus states, and hidden error message spans. Error messages are hidden by default using display: none and shown by adding the class 'show' via JavaScript.

### CSS Code

```
body {
  font-family: Arial, sans-serif;
  background: #f4f4f4;
```

```
  display: flex;
  justify-content: center;
  padding: 2rem;
}

.form-group { margin-bottom: 1rem; }

label { display: block; font-weight: bold; margin-bottom: 4px; }

input {
  width: 100%;
  padding:  0.6rem  0.8rem;
  border: 1px solid #ccc;
  border-radius:      6px;
  font-size: 1rem;
}

input:focus { border-color: #4a90e2; outline: none; }
input.is-valid  { border-color: green; }
input.is-error  { border-color: red;    }

.error-msg {
  display: none;
  color: red;
  font-size: 0.8rem;
  margin-top: 4px;
}
.error-msg.show { display: block; }

button {
  width:          100%;
  padding:     0.75rem;
  background:  #4a90e2;
  color:         white;
  border:         none;
  border-radius:   6px;
  font-size:      1rem;
  cursor: pointer;
}
```

## Output Screen – Step 2:




The form is now styled with a dark card layout, labelled fields, icon placeholders, and a styled submit button. Error message spans are present in the DOM but not visible. No validation logicis active yet.

## Step 3: Adding JavaScript Validation with Validator.js

LoadValidator.jsfromCDNand writeJavaScriptto validateeach field on blur and input events. On submit, all fields are checked and error messages are shown for any that fail validation.

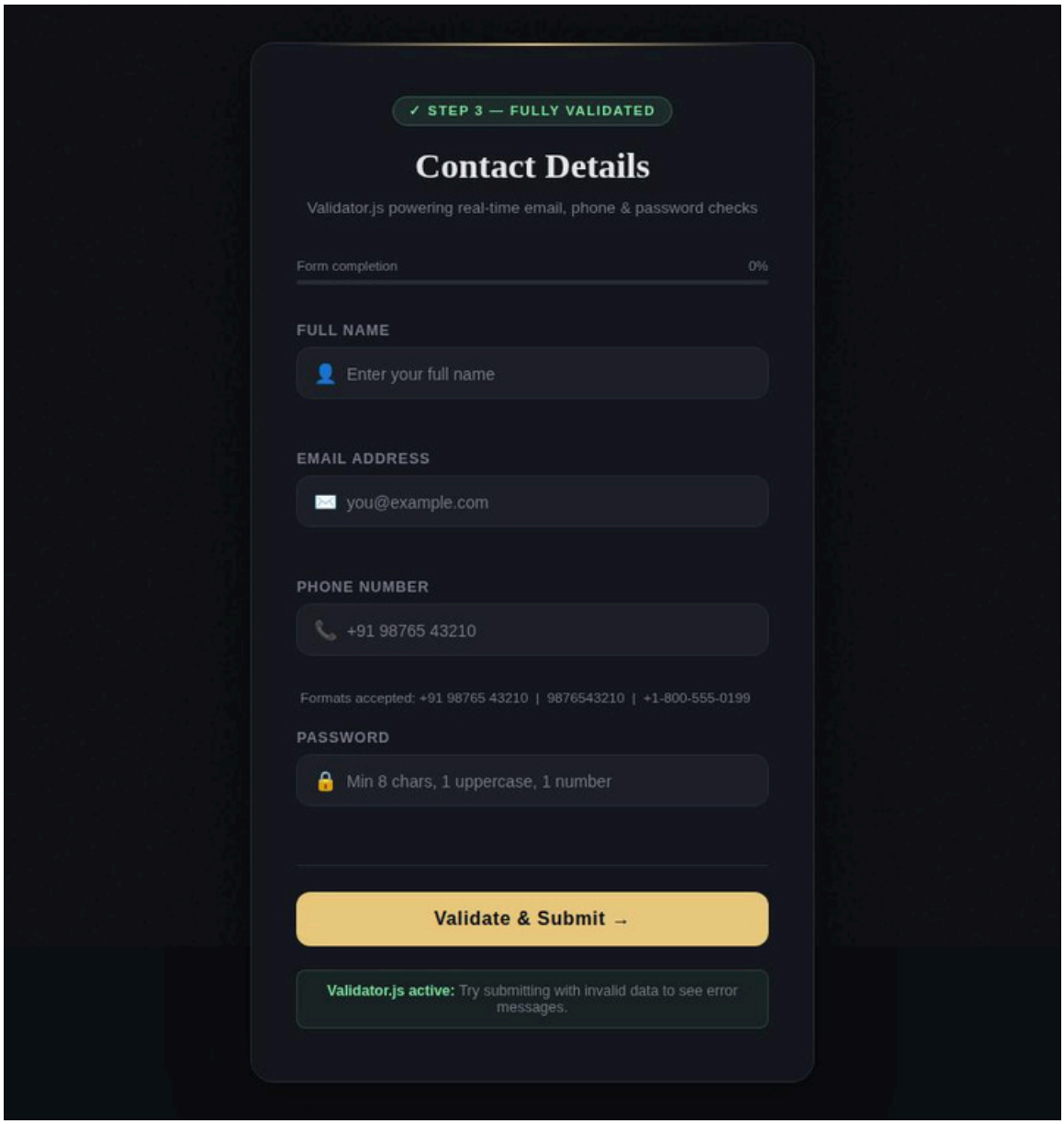
### Loading Validator.js

```
<scriptsrc="https://cdnjs.cloudflare.com/ajax/libs/
validator/13.12.0/validator.min.js"></script>
```

## JavaScript Code

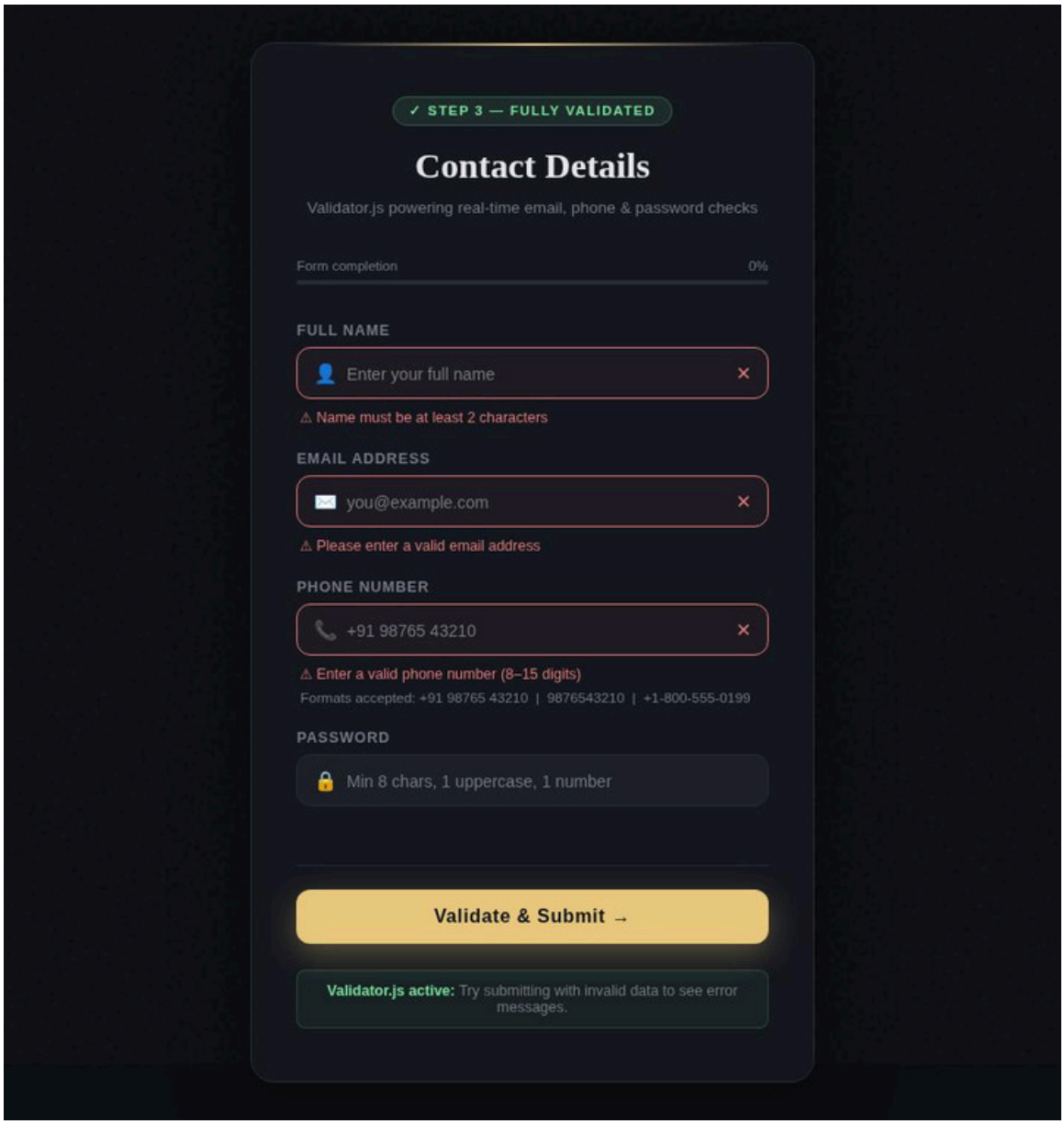
```
function validateField(name) {
  const val = document.getElementById(name).value.trim();
  if (name === "fullName")
    return val.length >= 2 && /^[A-Za-z\s]+$/.test(val);
  if (name === "email")
    return val !== "" && validator.isEmail(val);
  if (name === "phone") {
    const s = val.replace(/[\s\-()]/g, "");
    return validator.isMobilePhone(s, "any");
  }
  if (name === "password")
    return validator.isStrongPassword(val, {
      minLength: 8, minUppercase: 1, minNumbers: 1, minSymbols: 0
    });
}

function applyState(name, isValid) {
  const input = document.getElementById(name);
  const error = document.getElementById(name + "Error");
  input.classList.toggle("is-valid", isValid);
  input.classList.toggle("is-error", !isValid);
  !isValid);
  error.classList.toggle("show",
}

["fullName","email","phone","password"].forEach(name => {
  const input = document.getElementById(name);
  let touched = false;
  input.addEventListener("blur", () => {
    touched = true;
    applyState(name, validateField(name));
  });
  input.addEventListener("input", () => {
    if (touched) applyState(name, validateField(name));
  });
});

document.getElementById("validationForm")
.addEventListener("submit", function(e) {
    e.preventDefault();
    let allValid = true;
    ["fullName","email","phone","password"].forEach(name => {
      const ok = validateField(name);
      applyState(name, ok);
      if (!ok) allValid = false;
    });
    if (allValid) alert("Form submitted successfully!");
  });
```

## Output Screen – Step 3a: Form loaded (no input yet)



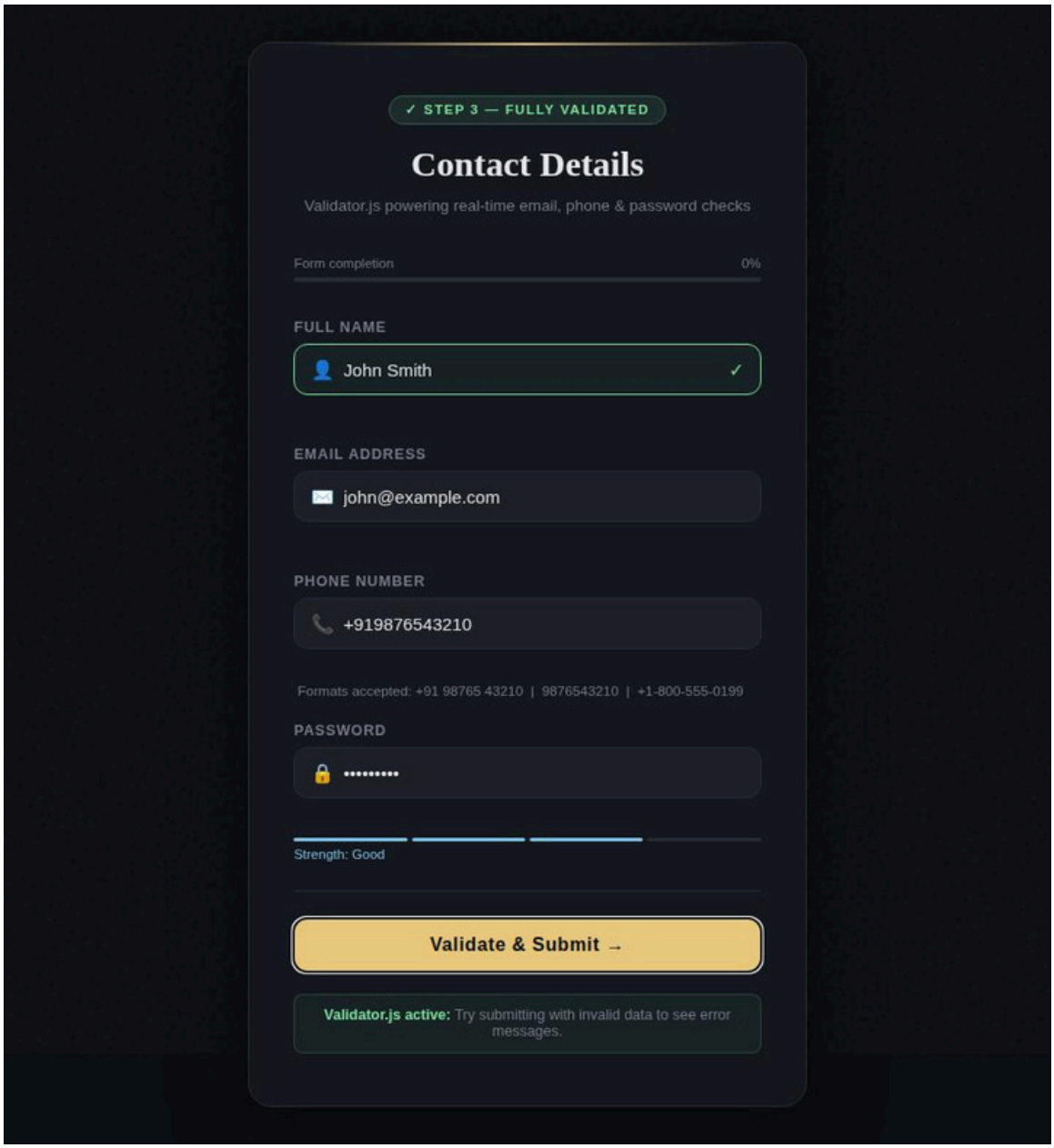

The styled form is displayed with all fields empty. The progress bar shows0%. No validation messages are visible as the user has not interacted with the form yet.

**Output Screen – Step 3b: Form submitted with invalid / empty fields**




When the Submit button is clicked with empty or invalid fields, all fields are validated at once. Red borders and red error messages appear below each field that fails validation. Theicon on the right of each input shows a cross symbol.

**Output Screen – Step 3c: All fields filled with valid data**




When all fields contain valid data, each input shows a green border and a checkmark icon. The progress bar reaches 100%. Clicking Submit at this point triggers the success message.

## Result

The experiment was successfully completed. A three-step form validation system was built using HTML for structure, CSS for styling, and JavaScript with Validator.js for validation. The form correctly validates email format using validator.isEmail(), phone numbers using validator.isMobilePhone(), and password strength using validator.isStrongPassword(), and displays appropriate error messages for invalid inputs.